PYTHON ACTIVITY-BASED LEARNING REPORT

Name: Rajesh	**Roll No:** 26000214
Branch: CSE	**Course:** Python Programming

Case 1: Progressive Electricity Billing

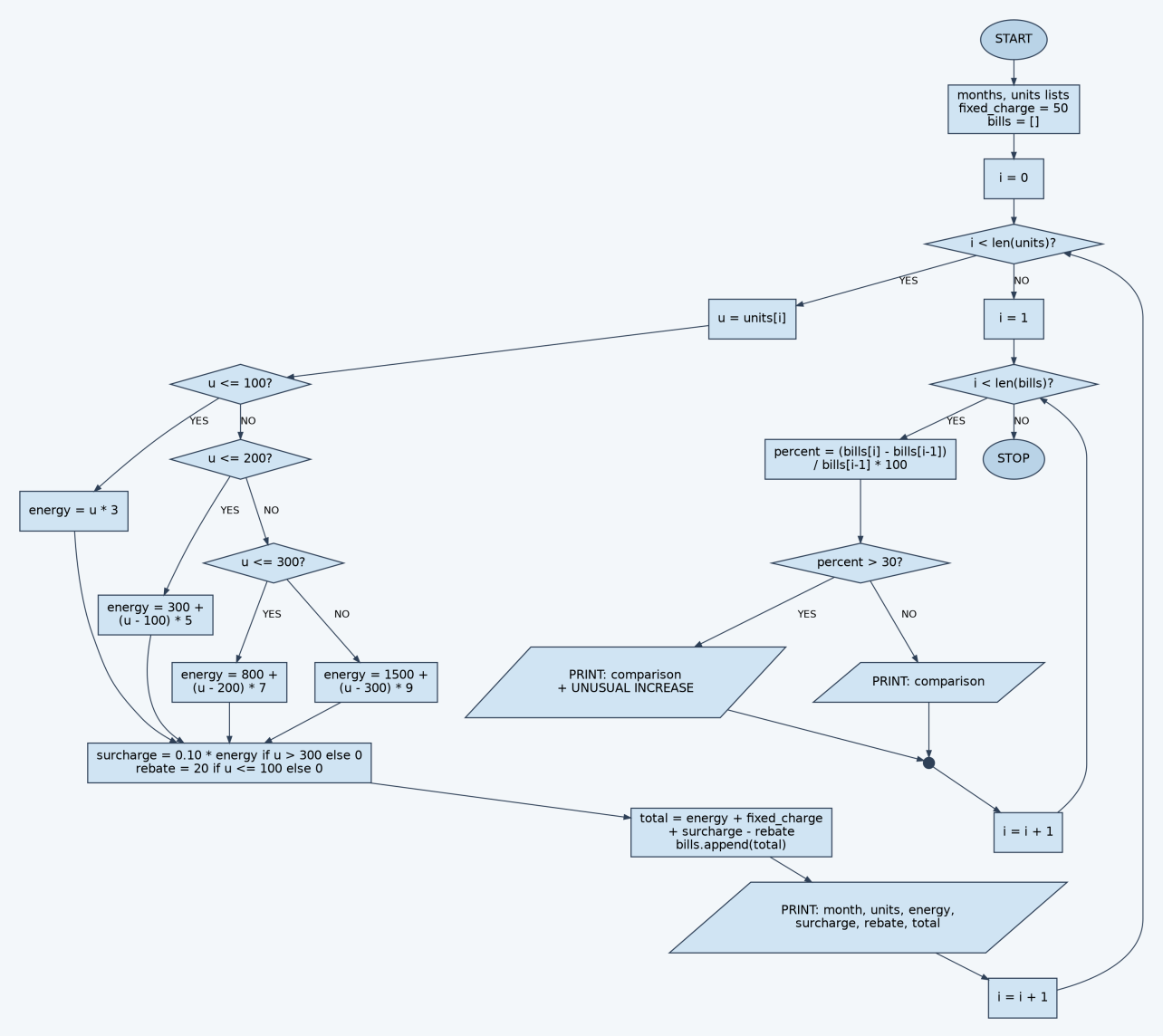
Problem Statement

Write a Python program to calculate the monthly electricity bills of a household for six months using progressive slab rates (Rs. 3 per unit for the first 100 units, Rs. 5 for the next 100, Rs. 7 for the next 100 and Rs. 9 for every unit above 300). A fixed charge of Rs. 50 is added to every bill, a 10% surcharge is added on the energy charge if the units exceed 300, and a rebate of Rs. 20 is given if the units are 100 or fewer. Then compare consecutive months and flag any month in which the bill rises by more than 30%.

Algorithm

- Step 1: Start.
- Step 2: Store the months and units in lists, set fixed_charge = 50 and create an empty list bills.
- Step 3: For each month, take the units u from the list.
- Step 4: If u ≤ 100, energy = u × 3; else if u ≤ 200, energy = 300 + (u - 100) × 5; else if u ≤ 300, energy = 800 + (u - 200) × 7; else energy = 1500 + (u - 300) × 9.
- Step 5: If u > 300, surcharge = 10% of energy, otherwise surcharge = 0. If u ≤ 100, rebate = 20, otherwise rebate = 0.
- Step 6: total = energy + fixed_charge + surcharge - rebate; print the row and store total in bills.
- Step 7: For every pair of consecutive months, percent = (change ÷ previous bill) × 100.
- Step 8: Print the comparison; if percent > 30, also print the UNUSUAL INCREASE flag.
- Step 9: Stop.

Flowchart


START
months, units lists
fixed_charge = 50
bills = []
i = 0
i < len(units)?
YES
NO
u = units[i]
i = 1
u <= 100?
i < len(bills)?
YES
NO
YES
NO
u <= 200?
percent = (bills[i] - bills[i-1]) / bills[i-1] * 100
STOP
energy = u * 3
YES
NO
u <= 300?
percent > 30?
energy = 300 + (u - 100) * 5
YES
NO
YES
NO
energy = 800 + (u - 200) * 7
energy = 1500 + (u - 300) * 9
PRINT: comparison + UNUSUAL INCREASE
PRINT: comparison
surcharge = 0.10 * energy if u > 300 else 0
rebate = 20 if u <= 100 else 0
total = energy + fixed_charge + surcharge - rebate
bills.append(total)
i = i + 1
PRINT: month, units, energy, surcharge, rebate, total
i = i + 1

Source Code & Output

```
# Slabs: first 100 units Rs3, next 100 Rs5, next 100 Rs7, above 300 Rs9
# Fixed charge Rs50, surcharge 10% of energy charge if units > 300
# Rebate Rs20 if units <= 100, unusual increase = more than 30% rise over previous month

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
units = [90, 135, 150, 260, 140, 320]
fixed_charge = 50
bills = []

print("Month  Units  Energy  Surcharge  Rebate  Total Bill")
for i in range(len(units)):
    u = units[i]
    if u <= 100:
        energy = u * 3
    elif u <= 200:
        energy = 300 + (u - 100) * 5
    elif u <= 300:
        energy = 300 + 500 + (u - 200) * 7
    else:
        energy = 300 + 500 + 700 + (u - 300) * 9

    surcharge = 0
    if u > 300:
        surcharge = 0.10 * energy

    rebate = 0
    if u <= 100:
        rebate = 20

    total = energy + fixed_charge + surcharge - rebate
    bills.append(total)
    print(months[i], "  ", u, "  ", energy, "  ", surcharge, "  ", rebate, "  ", round(total, 2))

print("\nMonth-to-month comparison")
for i in range(1, len(bills)):
    change = bills[i] - bills[i - 1]
    percent = change / bills[i - 1] * 100
    line = months[i - 1] + " -> " + months[i] + ": " + str(round(percent, 2)) + "%"
    if percent > 30:
        line = line + "  <-- UNUSUAL INCREASE"
    print(line)
```

```
Month  Units  Energy  Surcharge  Rebate  Total Bill
Jan    90    270    0    20    300
Feb    135    475    0    0    525
Mar    150    550    0    0    600
Apr    260    1220    0    0    1270
May    140    500    0    0    550
Jun    320    1680    168.0    0    1898.0

Month-to-month comparison
Jan -> Feb: 75.0%  <-- UNUSUAL INCREASE
Feb -> Mar: 14.29%
Mar -> Apr: 111.67%  <-- UNUSUAL INCREASE
Apr -> May: -56.69%
May -> Jun: 245.09%  <-- UNUSUAL INCREASE
```

Case 2: ATM Transaction Sequence Simulation

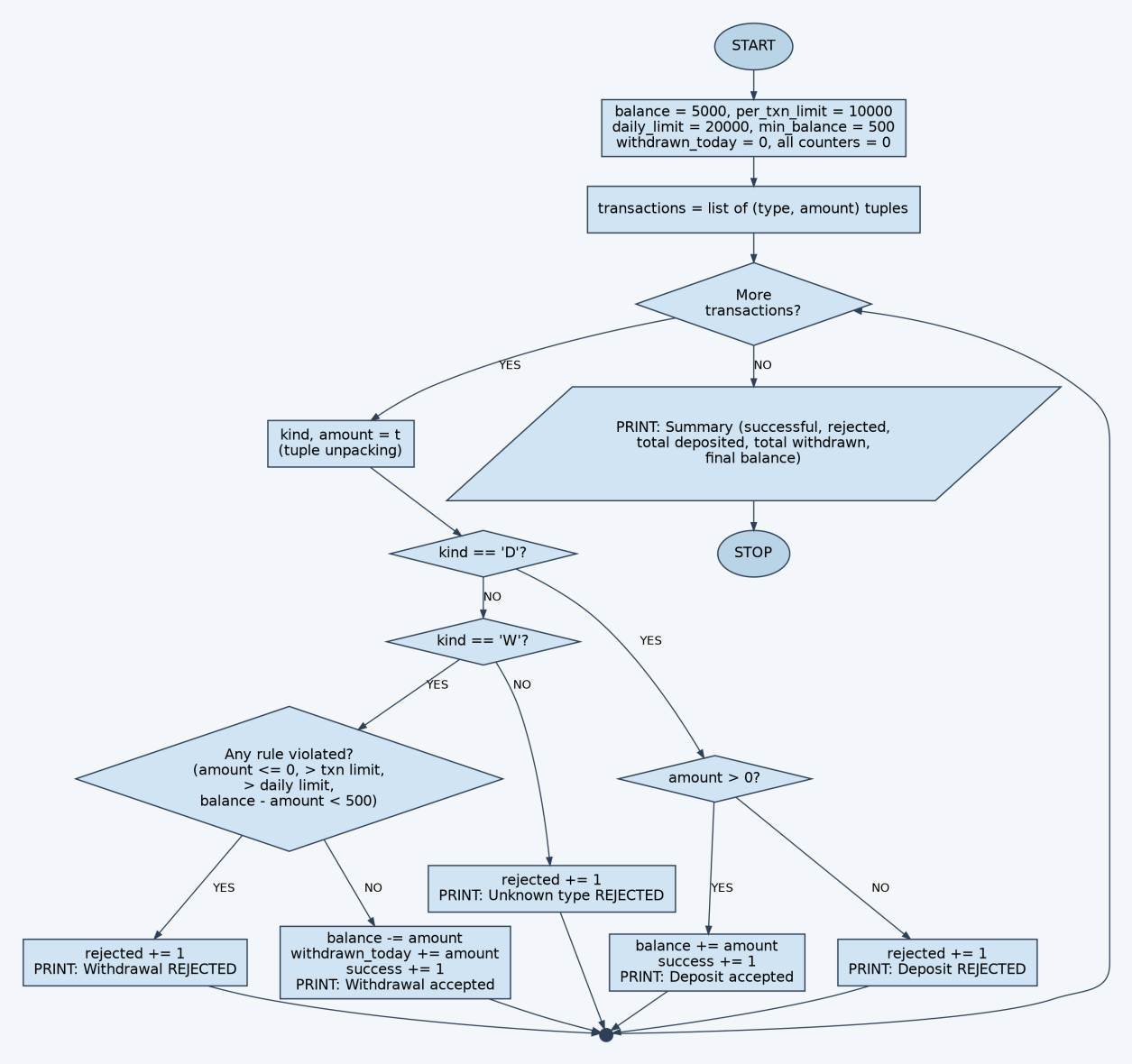
Problem Statement

Simulate an ATM that processes a sequence of deposits and withdrawals on an opening balance of Rs. 5000. A deposit must be a positive amount. A withdrawal is rejected if it is not positive, exceeds Rs. 10000 per transaction, takes the day's withdrawals above Rs. 20000, or leaves less than Rs. 500 in the account. Unknown transaction types are rejected, and a transaction summary is printed at the end.

Algorithm

- Step 1: Start.
- Step 2: Set balance = 5000, per_txn_limit = 10000, daily_limit = 20000, min_balance = 500, withdrawn_today = 0 and all counters to 0.
- Step 3: Store the transactions as a list of tuples (type, amount).
- Step 4: For each tuple, unpack it into kind and amount.
- Step 5: If kind is "D": accept the deposit and add it to the balance when amount > 0; otherwise reject it.
- Step 6: If kind is "W": reject when amount ≤ 0, amount > 10000, withdrawn_today + amount > 20000 or balance - amount < 500; otherwise subtract it from the balance and update withdrawn_today.
- Step 7: For any other kind, reject the transaction as an unknown type.
- Step 8: After the last transaction, print the number of successful and rejected transactions, total deposited, total withdrawn and the final balance. Stop.

Flowchart



Source Code & Output

```
# Rules: max Rs10000 per withdrawal, max Rs20000 withdrawn per day,
# minimum balance Rs500 must remain, deposits must be positive

balance = 5000
per_txn_limit = 10000
daily_limit = 20000
min_balance = 500
withdrawn_today = 0

transactions = [("D", 2000), ("W", 3000), ("W", 12000), ("D", -500),
                ("W", 4500), ("X", 100), ("D", 15000), ("W", 9000), ("W", 9000)]

success = 0
rejected = 0
total_dep = 0
total_wd = 0

for t in transactions:
    kind, amount = t          # tuple unpacking
    if kind == "D":
        if amount > 0:
            balance = balance + amount
            total_dep = total_dep + amount
            success = success + 1
            print("Deposit", amount, "accepted. Balance:", balance)
        else:
            rejected = rejected + 1
            print("Deposit", amount, "REJECTED: invalid amount")
    elif kind == "W":
        if amount <= 0:
            rejected = rejected + 1
            print("Withdrawal", amount, "REJECTED: invalid amount")
        elif amount > per_txn_limit:
            rejected = rejected + 1
            print("Withdrawal", amount, "REJECTED: exceeds per-transaction limit")
        elif withdrawn_today + amount > daily_limit:
            rejected = rejected + 1
            print("Withdrawal", amount, "REJECTED: exceeds daily limit")
        elif balance - amount < min_balance:
            rejected = rejected + 1
            print("Withdrawal", amount, "REJECTED: insufficient balance")
        else:
            balance = balance - amount
            withdrawn_today = withdrawn_today + amount
            total_wd = total_wd + amount
            success = success + 1
            print("Withdrawal", amount, "accepted. Balance:", balance)
    else:
        rejected = rejected + 1
        print("Transaction type", kind, "REJECTED: unknown type")

print("\n--- Summary ---")
print("Successful:", success, " Rejected:", rejected)
print("Total deposited:", total_dep, " Total withdrawn:", total_wd)
print("Final balance:", balance)
```

```
Deposit 2000 accepted. Balance: 7000
Withdrawal 3000 accepted. Balance: 4000
Withdrawal 12000 REJECTED: exceeds per-transaction limit
Deposit -500 REJECTED: invalid amount
Withdrawal 4500 REJECTED: insufficient balance
Transaction type X REJECTED: unknown type
Deposit 15000 accepted. Balance: 19000
Withdrawal 9000 accepted. Balance: 10000
Withdrawal 9000 REJECTED: exceeds daily limit

--- Summary ---
Successful: 4  Rejected: 5
Total deposited: 17000  Total withdrawn: 12000
Final balance: 10000
```

Case 3: Hollow Diamond Numeric Pattern

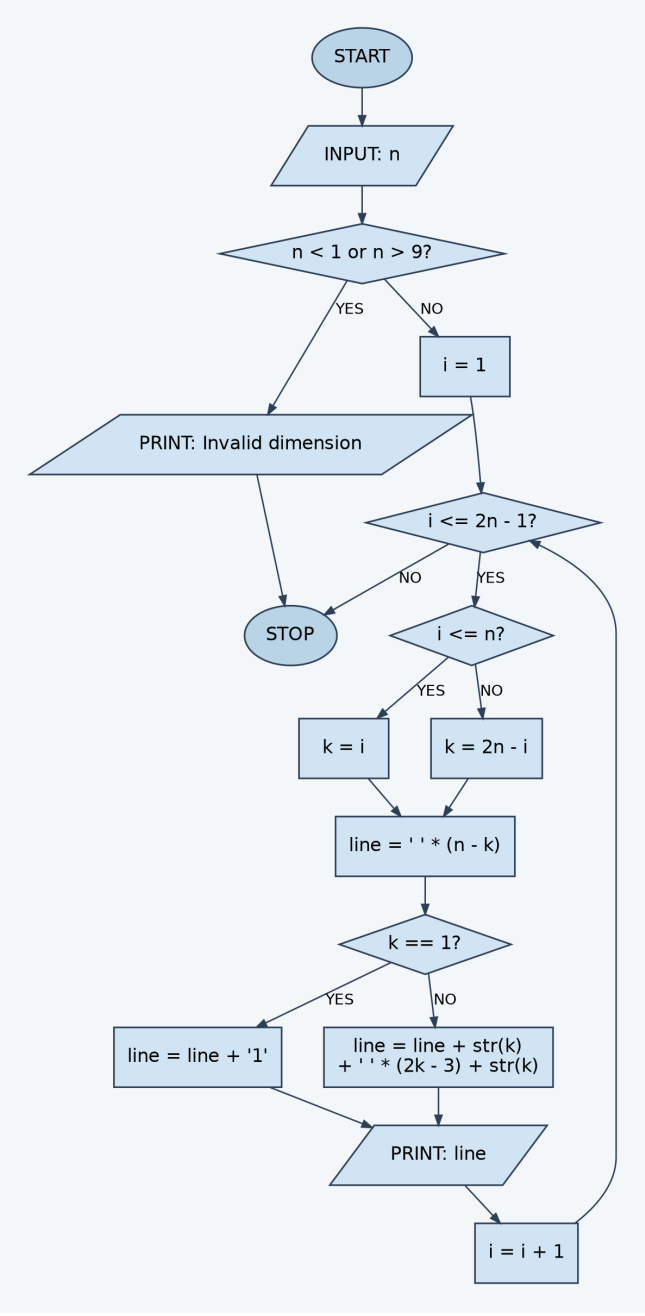
Problem Statement

Write a Python program to print a hollow, symmetric numeric diamond of the size entered by the user. Each row shows its row number at the left and right edges with spaces in between, the output must be carefully aligned, and invalid sizes (below 1 or above 9) must be rejected with a message.

Algorithm

- Step 1: Start.
- Step 2: Read the size n from the user.
- Step 3: If n < 1 or n > 9, print "Invalid dimension" and stop.
- Step 4: Repeat for i = 1 to 2n - 1.
- Step 5: If i ≤ n, k = i; otherwise k = 2n - i.
- Step 6: Start the line with (n - k) spaces.
- Step 7: If k = 1, add "1" to the line; otherwise add k, then (2k - 3) spaces, then k again.
- Step 8: Print the line and go to the next i. Stop.

Flowchart



Source Code & Output

```
n = int(input("Enter size of diamond (1 to 9): "))

if n < 1 or n > 9:
    print("Invalid dimension. Enter a value between 1 and 9.")
else:
    for i in range(1, 2 * n):
        if i <= n:
            k = i
        else:
            k = 2 * n - i
        line = " " * (n - k)
        if k == 1:
            line = line + "1"
        else:
            line = line + str(k) + " " * (2 * k - 3) + str(k)
        print(line)
```

```
Enter size of diamond (1 to 9): 4
   1
  2 2
 3   3
4     4
 3   3
  2 2
   1
```

Case 4: Password Policy Validation Engine

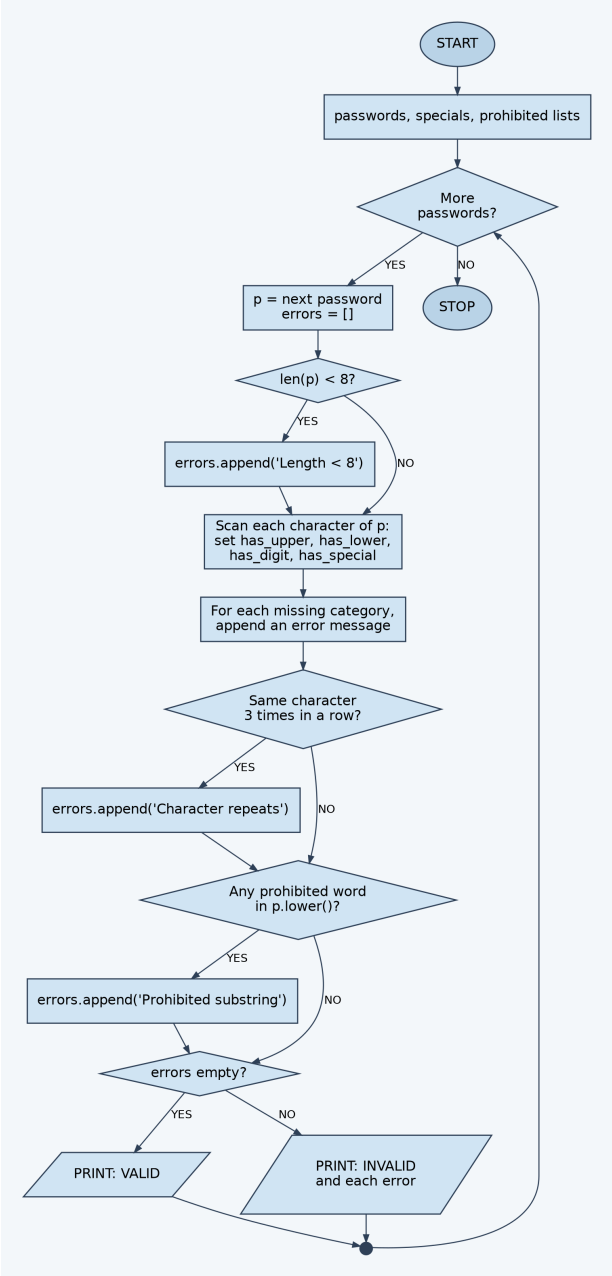
Problem Statement

Validate a list of sample passwords against a policy: minimum length 8, at least one uppercase letter, one lowercase letter, one digit and one special character (!@#$%^&*), no character repeated three times in a row, and no prohibited substring (password, admin, 1234, qwerty). For every invalid password, each failed condition must be explained.

Algorithm

- Step 1: Start.
- Step 2: Store the passwords, the allowed special characters and the prohibited words in lists/strings.
- Step 3: For each password p, create an empty list errors.
- Step 4: If len(p) < 8, add a length error.
- Step 5: Scan every character of p and note whether an uppercase, lowercase, digit and special character is present.
- Step 6: For every missing category, add an error message.
- Step 7: If any character repeats three times consecutively, add a repetition error.
- Step 8: If any prohibited word occurs in p (in lower case), add a prohibited-substring error.
- Step 9: If errors is empty print VALID; otherwise print INVALID followed by every error. Stop.

Flowchart



Source Code & Output

```
# Rules: length >= 8, upper, lower, digit, special (!@#$%^&*),
# no character repeated 3 times in a row, no prohibited substrings

passwords = ["Hello@123", "password1", "ABC12", "Aaa@1234x", "Admin#2024", "Strong$Pass9"]
specials = "!@#$%^&*"
prohibited = ["password", "admin", "1234", "qwerty"]

for p in passwords:
    errors = []
    if len(p) < 8:
        errors.append("Length is less than 8 characters")

    has_upper = False
    has_lower = False
    has_digit = False
    has_special = False
    for ch in p:
        if ch.isupper():
            has_upper = True
        elif ch.islower():
            has_lower = True
        elif ch.isdigit():
            has_digit = True
        elif ch in specials:
            has_special = True

    if not has_upper:
        errors.append("Missing uppercase letter")
    if not has_lower:
        errors.append("Missing lowercase letter")
    if not has_digit:
        errors.append("Missing digit")
    if not has_special:
        errors.append("Missing special character (!@#$%^&*)")

    for i in range(len(p) - 2):
        if p[i] == p[i + 1] and p[i + 1] == p[i + 2]:
            errors.append("Character '" + p[i] + "' repeats 3 times consecutively")
            break

    for word in prohibited:
        if word in p.lower():
            errors.append("Contains prohibited substring '" + word + "'")

    print("Password:", p)
    if len(errors) == 0:
        print("  VALID")
    else:
        print("  INVALID")
        for e in errors:
            print("   -", e)
```

```
Password: Hello@123
  VALID
Password: password1
  INVALID
   - Missing uppercase letter
   - Missing special character (!@#$%^&*)
   - Contains prohibited substring 'password'
Password: ABC12
  INVALID
   - Length is less than 8 characters
   - Missing lowercase letter
   - Missing special character (!@#$%^&*)
Password: Aaa@1234x
  INVALID
   - Contains prohibited substring '1234'
Password: Admin#2024
  INVALID
   - Contains prohibited substring 'admin'
Password: Strong$Pass9
  VALID
```

Case 5: String-Based Examination Answer Evaluation

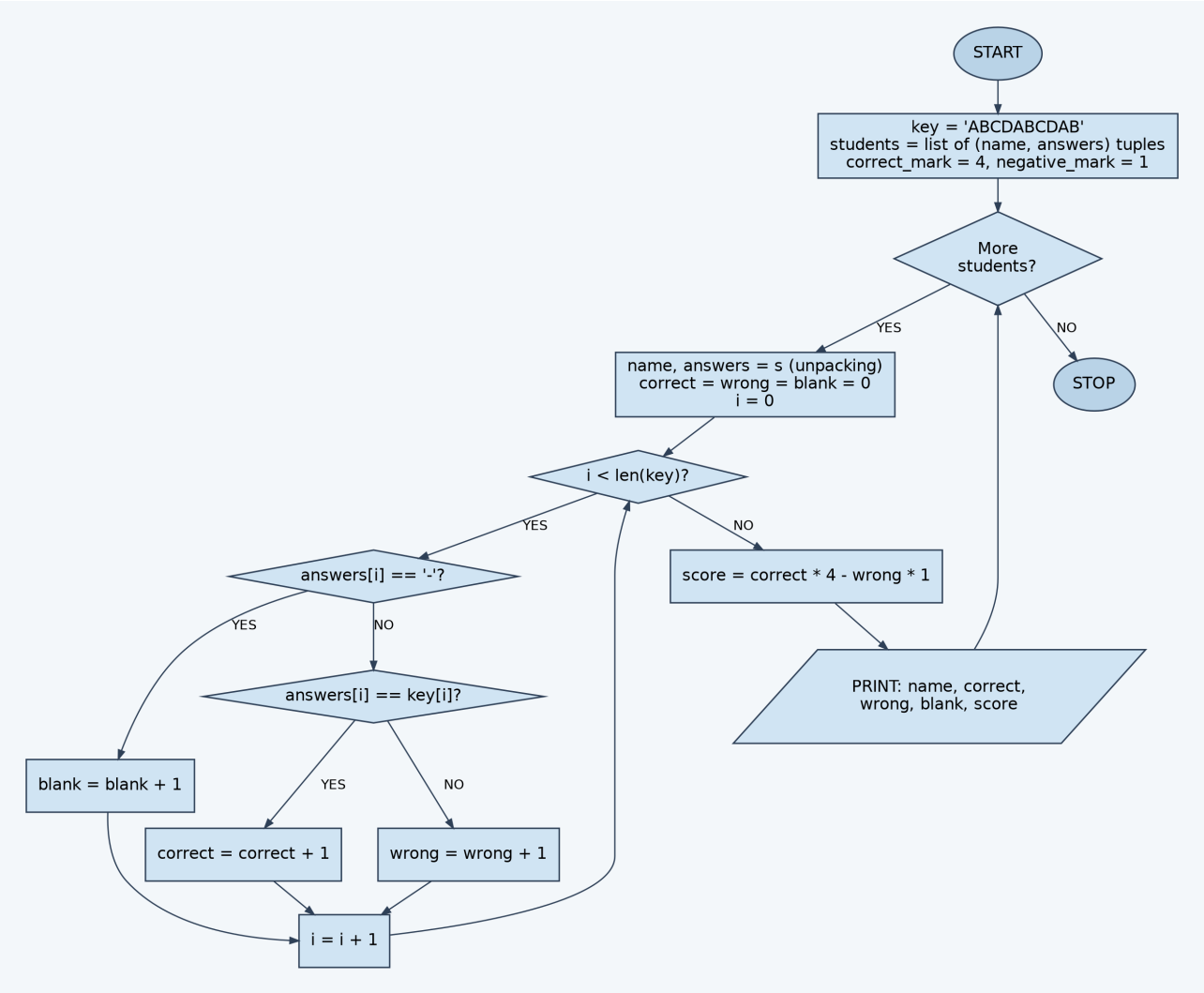
Problem Statement

Compare each student's answer string with the answer key and count the correct, incorrect and unanswered responses ("-" means unanswered). Award +4 marks for every correct answer, deduct 1 mark for every wrong answer, give 0 for unanswered questions and produce a student-wise result table.

Algorithm

- Step 1: Start.
- Step 2: Store the answer key, the students as tuples (name, answers) and set correct_mark = 4, negative_mark = 1.
- Step 3: For each student, unpack the tuple into name and answers and set correct = wrong = blank = 0.
- Step 4: For every position i of the key, if answers[i] is "-", increase blank.
- Step 5: Else if answers[i] equals key[i], increase correct; otherwise increase wrong.
- Step 6: score = correct × 4 - wrong × 1.
- Step 7: Print the name, correct, wrong, unanswered and score for the student.
- Step 8: Repeat for all students. Stop.

Flowchart



Source Code & Output

```
# '-' means unanswered. +4 for correct, -1 for wrong, 0 for unanswered

key = "ABCDABCDAB"
students = [("Ravi", "ABCDABCDAB"),
            ("Sita", "ABC-ABDDA-"),
            ("Kiran", "DCBADCBADC"),
            ("Anu", "ABCDAB----")]
correct_mark = 4
negative_mark = 1

print("Name    Correct  Wrong  Unanswered  Score")
for s in students:
    name, answers = s          # tuple unpacking
    correct = 0
    wrong = 0
    blank = 0
    for i in range(len(key)):
        if answers[i] == "-":
            blank = blank + 1
        elif answers[i] == key[i]:
            correct = correct + 1
        else:
            wrong = wrong + 1
    score = correct * correct_mark - wrong * negative_mark
    print(name, "   ", correct, "   ", wrong, "   ", blank, "   ", score)
```

```
Name    Correct  Wrong  Unanswered  Score
Ravi     10     0     0     40
Sita     7     1     2     27
Kiran     0     10     0     -10
Anu     6     0     4     24
```

Case 6: Text Compression Using Character Runs

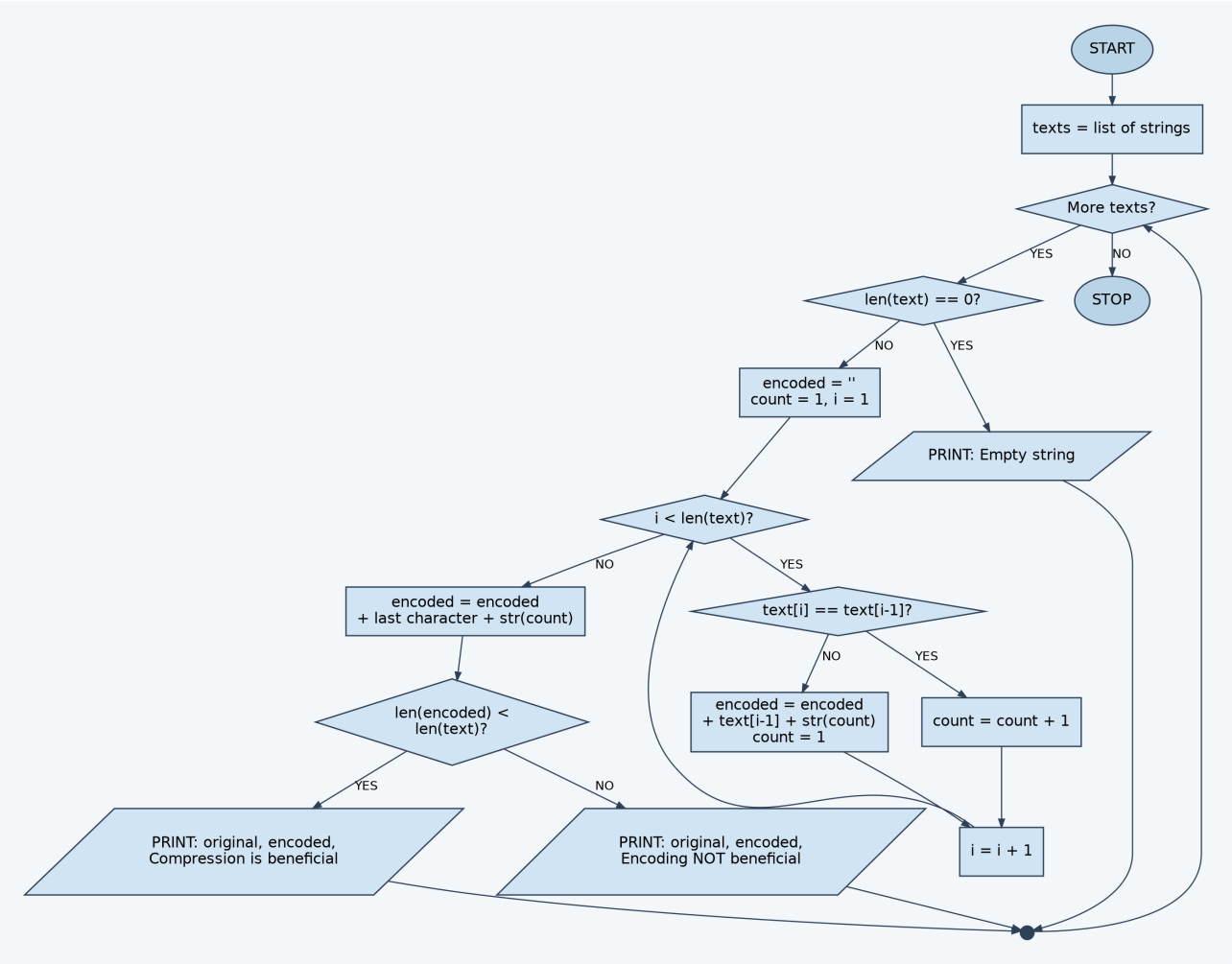
Problem Statement

Encode a string by grouping consecutive repeated characters (for example, "aaabbb" becomes "a3b3"), then compare the lengths of the original and the encoded strings and report whether encoding is beneficial. Empty strings and strings where encoding does not shorten the text must also be handled.

Algorithm

- Step 1: Start.
- Step 2: Store the sample strings in a list.
- Step 3: For each text, if its length is 0, print "Empty string" and move to the next text.
- Step 4: Set encoded = "", count = 1 and start from the second character (i = 1).
- Step 5: If text[i] equals text[i-1], increase count; otherwise append text[i-1] and count to encoded and reset count to 1.
- Step 6: After the loop, append the last character and its count to encoded.
- Step 7: Print the original and encoded strings with their lengths.
- Step 8: If len(encoded) < len(text), print that compression is beneficial; otherwise print that encoding is not beneficial. Stop.

Flowchart



Source Code & Output

```
texts = ["aaabbbcccd", "abcd", "aaaaaaaaaa", "aabbaa", ""]

for text in texts:
    if len(text) == 0:
        print("Empty string - nothing to encode\n")
    else:
        encoded = ""
        count = 1
        for i in range(1, len(text)):
            if text[i] == text[i - 1]:
                count = count + 1
            else:
                encoded = encoded + text[i - 1] + str(count)
                count = 1
        encoded = encoded + text[len(text) - 1] + str(count)

        print("Original :", text, "(length", len(text), ")")
        print("Encoded  :", encoded, "(length", len(encoded), ")")
        if len(encoded) < len(text):
            print("Result   : Compression is beneficial")
        else:
            print("Result   : Encoding NOT beneficial, keep original")
        print()
```

```
Original : aaabbbcccd (length 10 )
Encoded  : a3b3c3d1 (length 8 )
Result   : Compression is beneficial

Original : abcd (length 4 )
Encoded  : a1b1c1d1 (length 8 )
Result   : Encoding NOT beneficial, keep original

Original : aaaaaaaaaa (length 10 )
Encoded  : a10 (length 3 )
Result   : Compression is beneficial

Original : aabbaa (length 6 )
Encoded  : a2b2a2 (length 6 )
Result   : Encoding NOT beneficial, keep original

Empty string - nothing to encode
```

Case 7: Nested Loop Calendar Generator

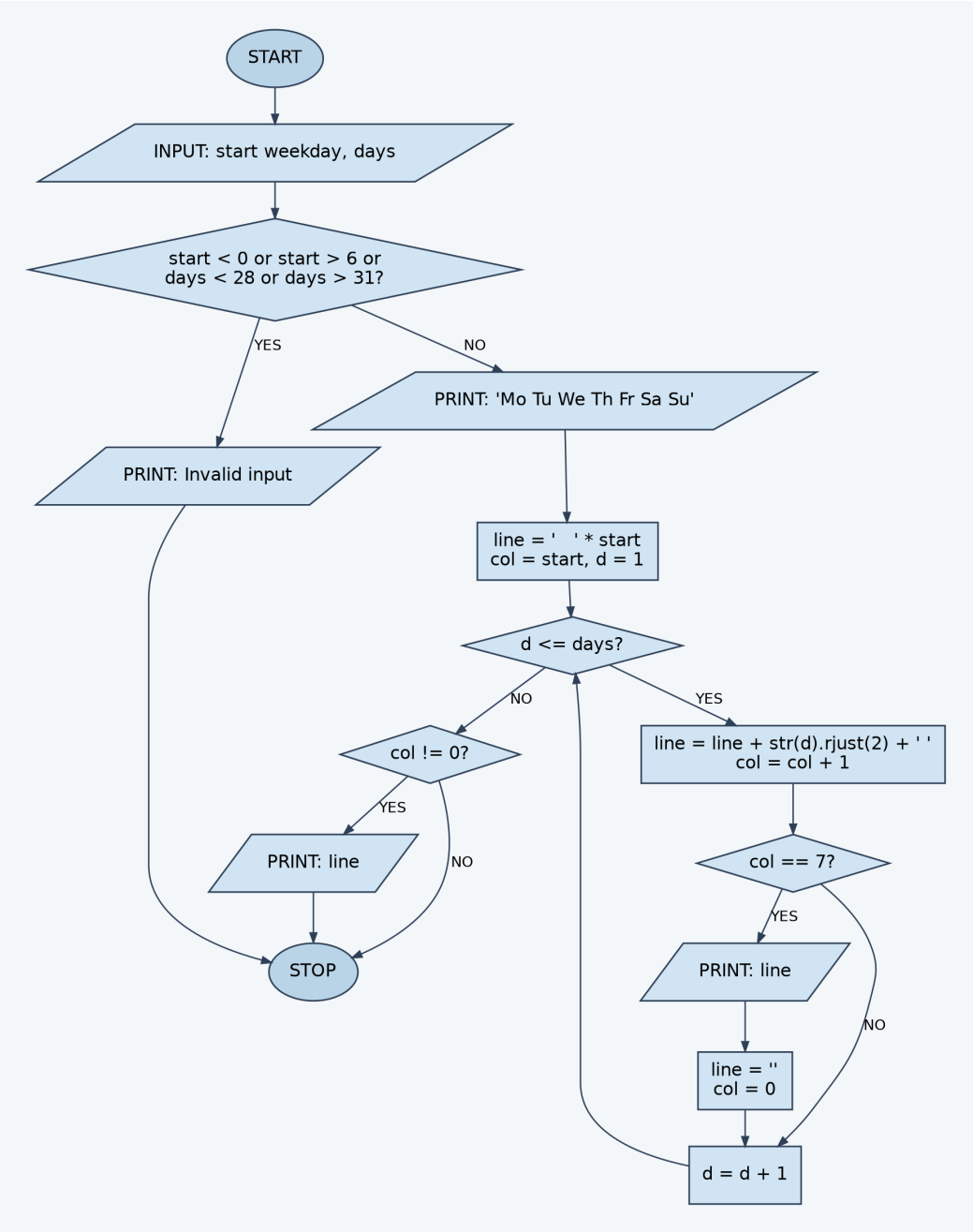
Problem Statement

Generate a formatted monthly calendar from the starting weekday (0 = Monday ... 6 = Sunday) and the number of days in the month. The dates must be aligned under the weekday headings, weeks must wrap correctly, and invalid inputs must be rejected.

Algorithm

- Step 1: Start.
- Step 2: Read the starting weekday and the number of days in the month.
- Step 3: If start < 0 or start > 6 or days < 28 or days > 31, print "Invalid input" and stop.
- Step 4: Print the heading "Mo Tu We Th Fr Sa Su".
- Step 5: Set line = 3 spaces × start and col = start.
- Step 6: For each date d from 1 to days, add d (right-aligned in 2 characters) and a space to line, and increase col.
- Step 7: When col reaches 7, print the line and reset line = "" and col = 0.
- Step 8: After the last date, if col ≠ 0, print the remaining line. Stop.

Flowchart



Source Code & Output

```
# Weekday numbering: 0=Mon, 1=Tue, ... 6=Sun
start = int(input("Starting weekday (0=Mon ... 6=Sun): "))
days = int(input("Number of days in month (28-31): "))

if start < 0 or start > 6 or days < 28 or days > 31:
    print("Invalid input")
else:
    print("Mo Tu We Th Fr Sa Su")
    line = "   " * start
    col = start
    for d in range(1, days + 1):
        line = line + str(d).rjust(2) + " "
        col = col + 1
        if col == 7:
            print(line)
            line = ""
            col = 0
    if col != 0:
        print(line)
```

```
Starting weekday (0=Mon ... 6=Sun): 3
Number of days in month (28-31): 30
Mo Tu We Th Fr Sa Su
          1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30
```

Case 8: Student Marks Matrix Analysis

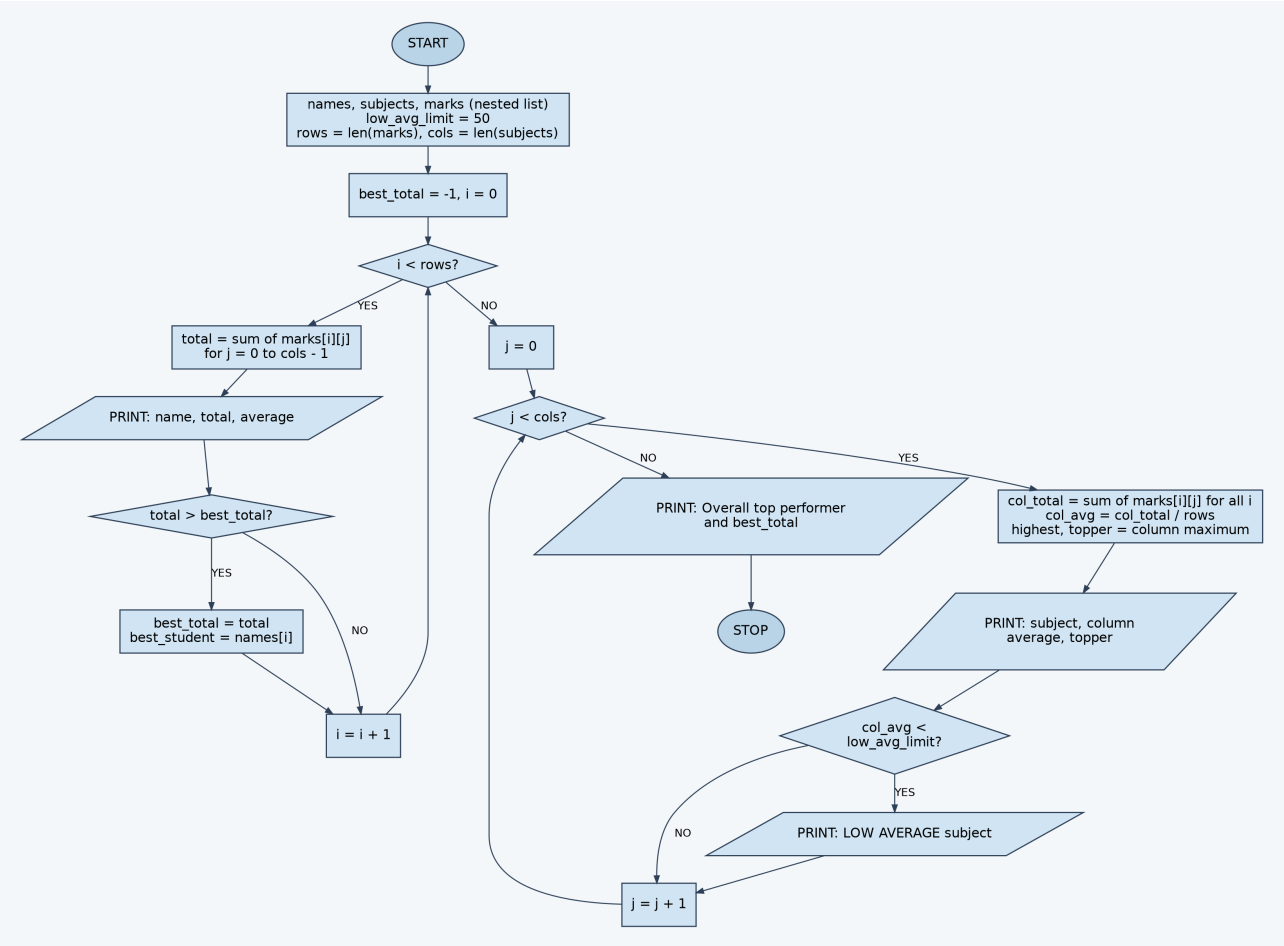
Problem Statement

Analyse a matrix (nested list) of marks of five students in four subjects. Calculate the total and average of every student (row-wise), the average and topper of every subject (column-wise), identify the overall top performer and detect subjects whose class average is below 50.

Algorithm

- Step 1: Start.
- Step 2: Store the names, subjects and the marks matrix (a nested list); set low_avg_limit = 50.
- Step 3: For each row i, add the marks of all subjects to get the total and compute the average.
- Step 4: Print the name, total and average; if the total exceeds best_total, update best_total and best_student.
- Step 5: For each column j, add the marks of all students to get col_total and compute col_avg.
- Step 6: While scanning the column, track the highest mark and the student who scored it (topper).
- Step 7: Print the subject, column average and topper; if col_avg < 50, print LOW AVERAGE subject.
- Step 8: Print the overall top performer and the best total. Stop.

Flowchart



Source Code & Output

```
names = ["Asha", "Bala", "Charan", "Divya", "Esha"]
subjects = ["Maths", "Physics", "Chemistry", "English"]
marks = [[85, 78, 92, 70],
         [45, 38, 50, 62],
         [90, 88, 79, 85],
         [60, 55, 48, 72],
         [30, 42, 35, 58]]
low_avg_limit = 50

rows = len(marks)
cols = len(subjects)

print("Row-wise statistics")
best_total = -1
best_student = ""
for i in range(rows):
    total = 0
    for j in range(cols):
        total = total + marks[i][j]
    avg = total / cols
    print(names[i], "Total:", total, "Average:", avg)
    if total > best_total:
        best_total = total
        best_student = names[i]

print("\nColumn-wise statistics")
for j in range(cols):
    col_total = 0
    highest = marks[0][j]
    topper = names[0]
    for i in range(rows):
        col_total = col_total + marks[i][j]
        if marks[i][j] > highest:
            highest = marks[i][j]
            topper = names[i]
    col_avg = col_total / rows
    print(subjects[j], "Average:", col_avg, "| Topper:", topper, "(", highest, ")")
    if col_avg < low_avg_limit:
        print("   -> LOW AVERAGE subject")

print("\nOverall top performer:", best_student, "with total", best_total)
```

```
Row-wise statistics
Asha Total: 325 Average: 81.25
Bala Total: 195 Average: 48.75
Charan Total: 342 Average: 85.5
Divya Total: 235 Average: 58.75
Esha Total: 165 Average: 41.25

Column-wise statistics
Maths Average: 62.0 | Topper: Charan ( 90 )
Physics Average: 60.2 | Topper: Charan ( 88 )
Chemistry Average: 60.8 | Topper: Asha ( 92 )
English Average: 69.4 | Topper: Charan ( 85 )

Overall top performer: Charan with total 342
```

Case 9: Tuple-Based Employee Shift Conflict Detection

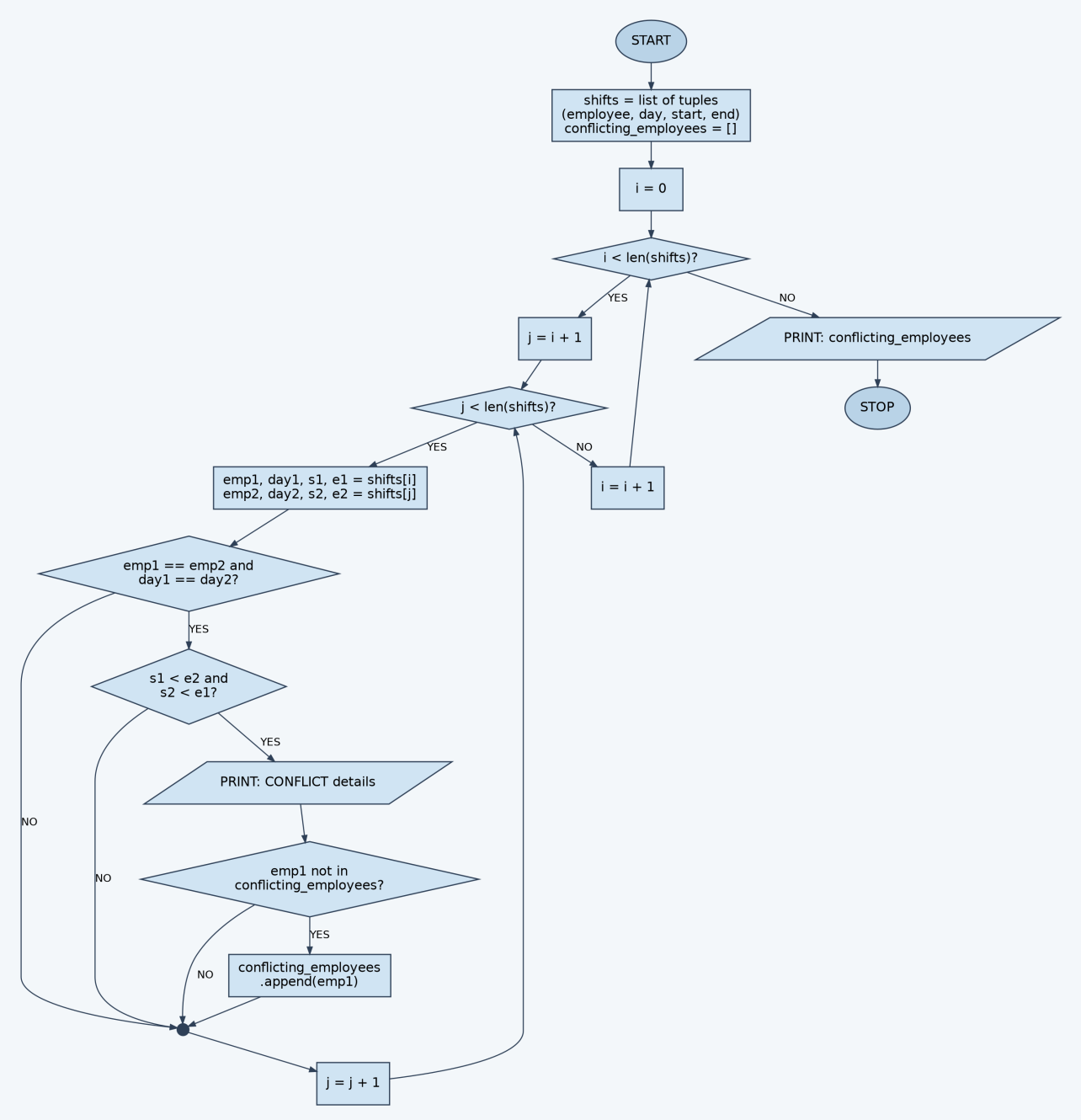
Problem Statement

Represent employee shifts as tuples (employee, day, start hour, end hour) in 24-hour format. Compare every pair of shifts, detect overlapping time intervals for the same employee on the same day and list the employees whose schedules conflict.

Algorithm

- Step 1: Start.
- Step 2: Store the shifts as a list of tuples and create an empty list conflicting_employees.
- Step 3: For every shift i, compare it with every later shift j.
- Step 4: Unpack both tuples into emp1, day1, s1, e1 and emp2, day2, s2, e2.
- Step 5: If the employee and the day are the same, check whether s1 < e2 and s2 < e1 (the intervals overlap).
- Step 6: If they overlap, print the CONFLICT details.
- Step 7: If emp1 is not already in conflicting_employees, append it.
- Step 8: After all pairs are checked, print the list of employees with conflicts (or "No conflicts found"). Stop.

Flowchart



Source Code & Output

```
# Shift tuple: (employee, day, start_hour, end_hour) in 24-hour format
shifts = [("Arun", "Mon", 9, 13),
          ("Arun", "Mon", 12, 17),
          ("Bhavya", "Mon", 9, 13),
          ("Bhavya", "Tue", 13, 18),
          ("Chetan", "Wed", 8, 12),
          ("Chetan", "Wed", 12, 16),
          ("Arun", "Tue", 10, 14),
          ("Chetan", "Wed", 11, 15)]

conflicting_employees = []

for i in range(len(shifts)):
    for j in range(i + 1, len(shifts)):
        emp1, day1, s1, e1 = shifts[i]    # tuple unpacking
        emp2, day2, s2, e2 = shifts[j]
        if emp1 == emp2 and day1 == day2:
            if s1 < e2 and s2 < e1:
                print("CONFLICT:", emp1, "on", day1, "->",
                      str(s1) + "-" + str(e1), "overlaps", str(s2) + "-" + str(e2))
                if emp1 not in conflicting_employees:
                    conflicting_employees.append(emp1)

print("\nEmployees with conflicting schedules:", conflicting_employees)
if len(conflicting_employees) == 0:
    print("No conflicts found")
```

```
CONFLICT: Arun on Mon -> 9-13 overlaps 12-17
CONFLICT: Chetan on Wed -> 8-12 overlaps 11-15
CONFLICT: Chetan on Wed -> 12-16 overlaps 11-15

Employees with conflicting schedules: ['Arun', 'Chetan']
```

Case 10: Tuple-Based Route Distance Analysis

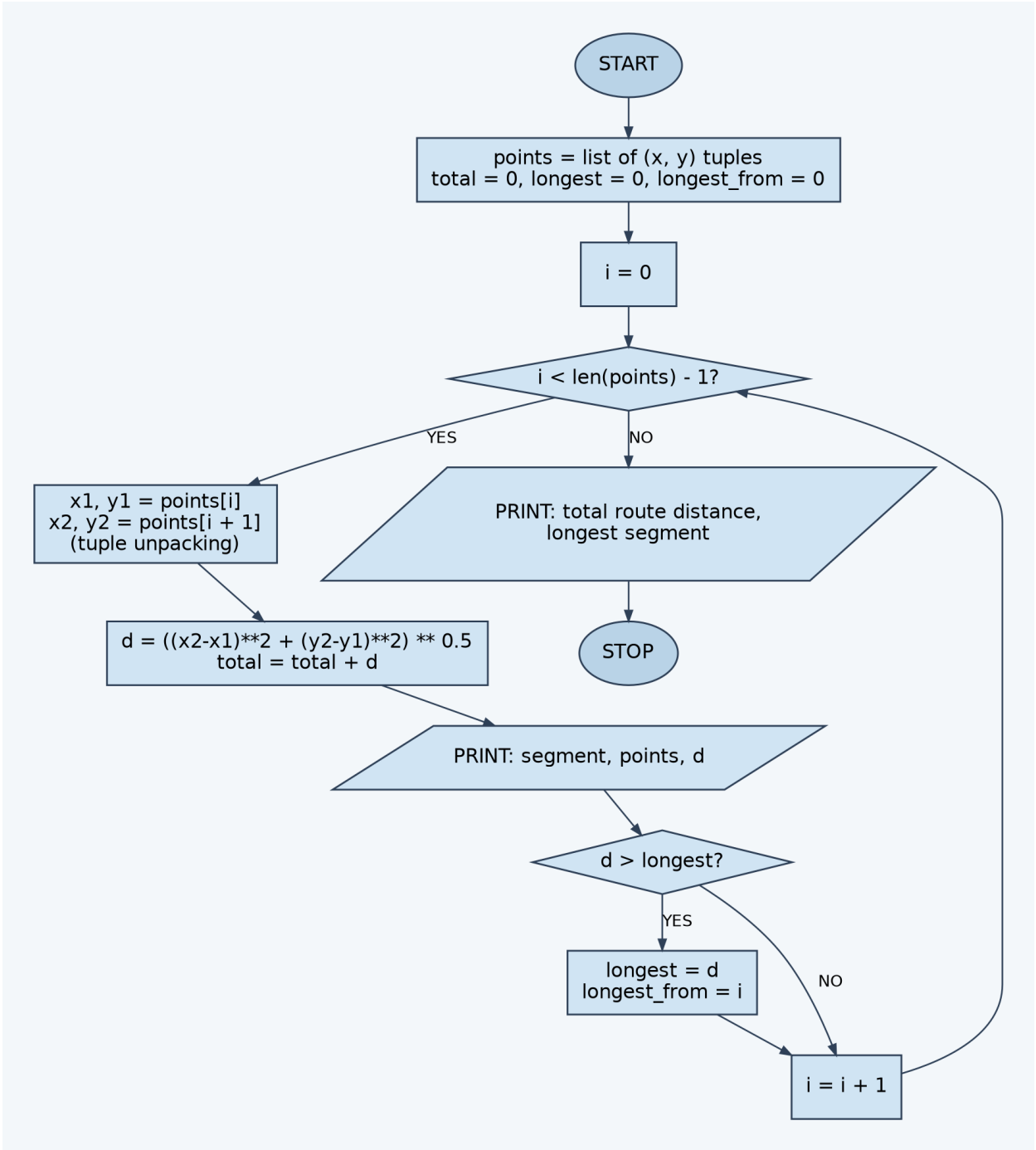
Problem Statement

Store geographical coordinates as tuples (x, y) in kilometres, calculate the distance between every pair of successive points using the distance formula sqrt((x2 - x1)² + (y2 - y1)²), find the total route distance and identify the longest route segment.

Algorithm

- Step 1: Start.
- Step 2: Store the points as a list of (x, y) tuples and set total = 0, longest = 0 and longest_from = 0.
- Step 3: For each i from 0 to len(points) - 2, unpack points[i] into x1, y1 and points[i+1] into x2, y2.
- Step 4: Calculate d = ((x2 - x1)² + (y2 - y1)²) ^ 0.5 and add it to total.
- Step 5: Print the segment number, the two points and the distance d.
- Step 6: If d > longest, set longest = d and longest_from = i.
- Step 7: Print the total route distance and the longest segment with its end points. Stop.

Flowchart



Source Code & Output

```
# Distance formula: sqrt((x2-x1)^2 + (y2-y1)^2), coordinates in km
points = [(0, 0), (3, 4), (6, 8), (6, 14), (10, 17)]

total = 0
longest = 0
longest_from = 0

for i in range(len(points) - 1):
    x1, y1 = points[i]          # tuple unpacking
    x2, y2 = points[i + 1]
    d = ((x2 - x1) ** 2 + (y2 - y1) ** 2) ** 0.5
    total = total + d
    print("Segment", i + 1, ":", points[i], "->", points[i + 1], "=", round(d, 2), "km")
    if d > longest:
        longest = d
        longest_from = i

print("\nTotal route distance:", round(total, 2), "km")
print("Longest segment: Segment", longest_from + 1, "from", points[longest_from],
      "to", points[longest_from + 1], "=", round(longest, 2), "km")
```

```
Segment 1 : (0, 0) -> (3, 4) = 5.0 km
Segment 2 : (3, 4) -> (6, 8) = 5.0 km
Segment 3 : (6, 8) -> (6, 14) = 6.0 km
Segment 4 : (6, 14) -> (10, 17) = 5.0 km

Total route distance: 21.0 km
Longest segment: Segment 3 from (6, 8) to (6, 14) = 6.0 km
```